Guía Práctica de Laboratorio: Pipeline de Ingesta, Exploración y Calidad de Datos

Asignatura: Minería de Datos

Unidad 2: Integración, Exploración y Preparación de Datos (*Data Engineering*)

Estudiante: Gino Maximiliano Bermúdez Santos

Duración Estimada: 120 minutos

Modalidad: Individual (Jupyter Notebook / Python)

Rúbrica de Evaluación (Puntaje Total: 10 Puntos)

Criterio	Descripción	Puntaje
Ingesta y Extracción SQL	Conexión correcta, ejecución de consultas estructuradas y carga limpia en DataFrame.	**1.5 pts**
Análisis Exploratorio (EDA)	Correcta generación e interpretación de gráficos (boxplot, correlación, asimetría).	**2.5 pts**
Tratamiento de Inconsistencias	Limpieza de strings, normalización de categorías y validación de reglas de negocio.	**2.0 pts**
Imputación Algorítmica	Uso correcto de estrategias de imputación (mediana / KNNImputer) con justificación estadística.	**2.0 pts**
Gestión de Outliers y Cierre	Implementación matemática del IQR, tratamiento de atípicos y persistencia del resultado limpio.	**2.0 pts**

---

1. Contexto del Problema y Dataset Sintético

En el rol de **ingenieros de datos**, procesaremos una base cruda de clientes y transacciones proveniente de una base de datos relacional.

A continuación, inicializamos el entorno con SQLite en memoria simulando las fallas de calidad típicas presentes en sistemas transaccionales legacy.

In [1]:

```
import sqlite3
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import KNNImputer

# Configuracion estetica de visualizacion
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.dpi'] = 120
pd.set_option('display.max_columns', 10)
pd.set_option('display.width', 1000)

print("Entorno y librerias inicializadas correctamente.")
```

```
Entorno y librerias inicializadas correctamente.
```

Fase 1: Extracción Relacional y Verificación Inicial (20 min)

Objetivo

Conectarse al gestor relacional mediante Python (`sqlite3` / `SQLAlchemy`), ejecutar consultas estructuradas y volcar el resultado en un `DataFrame` de pandas.

Entregable

Cargar la tabla completa verificando tipos de datos (`dtypes`) y dimensiones del dataset (`shape`).

In [2]:

```
# Creacion de base de datos relacional simulada (Tema 1)
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE clientes_crudo (
    id_cliente INTEGER PRIMARY KEY,
    edad REAL,
    ingreso_mensual REAL,
    ciudad TEXT,
    antiguedad_meses REAL,
    nivel_riesgo TEXT
);
""")

# Insercion de datos con anomalias intencionales segun especificacion
datos = [
    (1, 25.0, 1200.0, 'Guayaquil', 12.0, 'Bajo'),
    (2, 34.0, 2100.0, 'guayaquil', 24.0, 'Medio'),
    (3, -5.0, 1800.0, 'Santa Elena', 6.0, 'Bajo'),
    (4, 45.0, np.nan, 'SANTA ELENA', 48.0, 'Alto'),
    (5, 52.0, 95000.0, 'La Libertad', 60.0, 'Alto'),
    (6, np.nan, 1400.0, 'Salinas', 10.0, 'Bajo'),
    (7, 29.0, 1600.0, 'salinas', -2.0, 'Bajo'),
    (8, 41.0, 3100.0, 'Guayaquil', 36.0, 'Medio'),
    (9, 38.0, np.nan, 'Manta', 18.0, 'Bajo'),
    (10, 60.0, 3500.0, 'Guayaquil', 120.0, 'Alto'),
    (11, 22.0, 900.0, 'Salinas', 4.0, 'Bajo'),
    (12, 140.0, 2200.0, 'Santa Elena', 15.0, 'Medio'),
    (13, 31.0, 1950.0, 'La Libertad', 20.0, 'Bajo'),
    (14, 48.0, 4200.0, 'guayaquil', 50.0, 'Alto'),
    (15, 36.0, 2300.0, 'Santa Elena', 25.0, 'Medio')
]

cursor.executemany("INSERT INTO clientes_crudo VALUES (?, ?, ?, ?, ?, ?)", datos)
conn.commit()
print(f"Base de datos relacional inicializada. Registros insertados: {len(datos)}")
```

```
Base de datos relacional inicializada. Registros insertados: 15
```

In [3]:

```
# Extraccion mediante consulta SQL estructurada a un DataFrame de pandas
query_extraccion = """
SELECT
    id_cliente,
    edad,
    ingreso_mensual,
    ciudad,
    antiguedad_meses,
    nivel_riesgo
FROM clientes_crudo;
"""

df_crudo = pd.read_sql_query(query_extraccion, conn)

print("=" * 60)
print("VERIFICACION INICIAL DE DIMENSIONES Y TIPOS DE DATOS")
print("=" * 60)
print(f"Dimensiones del dataset (shape): {df_crudo.shape[0]} filas x {df_crudo.shape[1]} columnas")
print("\nTipos de datos detectados (dtypes):")
print(df_crudo.dtypes)
print("\nPrimeros registros extraidos:")
df_crudo
```

```
============================================================
VERIFICACION INICIAL DE DIMENSIONES Y TIPOS DE DATOS
============================================================
Dimensiones del dataset (shape): 15 filas x 6 columnas

Tipos de datos detectados (dtypes):
id_cliente            int64
edad                float64
ingreso_mensual     float64
ciudad                  str
antiguedad_meses    float64
nivel_riesgo            str
dtype: object

Primeros registros extraidos:
```

Out [3]:

	id_cliente	edad	ingreso_mensual	ciudad	antiguedad_meses	nivel_riesgo
0	1	25.0	1200.0	Guayaquil	12.0	Bajo
1	2	34.0	2100.0	guayaquil	24.0	Medio
2	3	-5.0	1800.0	Santa Elena	6.0	Bajo
3	4	45.0	NaN	SANTA ELENA	48.0	Alto
4	5	52.0	95000.0	La Libertad	60.0	Alto
5	6	NaN	1400.0	Salinas	10.0	Bajo
6	7	29.0	1600.0	salinas	-2.0	Bajo
7	8	41.0	3100.0	Guayaquil	36.0	Medio
8	9	38.0	NaN	Manta	18.0	Bajo
9	10	60.0	3500.0	Guayaquil	120.0	Alto
10	11	22.0	900.0	Salinas	4.0	Bajo
11	12	140.0	2200.0	Santa Elena	15.0	Medio
12	13	31.0	1950.0	La Libertad	20.0	Bajo
13	14	48.0	4200.0	guayaquil	50.0	Alto
14	15	36.0	2300.0	Santa Elena	25.0	Medio

Diagnóstico Inicial de la Ingesta

1. **Dimensiones:** El dataset posee **15 registros y 6 atributos**.
2. **Tipos de Datos:** Los identificadores y medidas numéricas se reconocen como numéricos (`int64`, `float64`), mientras que `ciudad` y `nivel_riesgo` son categóricos/texto (`object`).
3. **Anomalías Visibles a Simple Vista:**
4. Valores nulos (`NaN`) en `ingreso_mensual` (clientes 4 y 9) y `edad` (cliente 6).
5. Inconsistencias de capitalización en `ciudad`: `'Guayaquil'` vs `'guayaquil'`, `'Santa Elena'` vs `'SANTA ELENA'`, `'Salinas'` vs `'salinas'`.
6. Valores biológicamente imposibles en `edad`: cliente 3 con `-5.0` años y cliente 12 con `140.0` años.
7. Valores imposibles en `antiguedad_meses`: cliente 7 con `-2.0` meses.
8. Posible valor extremo atípico en `ingreso_mensual`: cliente 5 con `95000.0`.

Fase 2: Análisis Exploratorio de Datos - EDA (30 min)

Objetivo

Detectar distribuciones, asimetrías y dependencias entre variables.

Tareas:

1. Generar estadísticas descriptivas completas (`describe(include='all')`).
2. Identificar la asimetría (*skewness*) en la variable `ingreso_mensual`.
3. Construir diagramas de caja (*boxplots*) para `ingreso_mensual` y `edad`, además de un mapa de calor (*heatmap*) de correlación entre variables numéricas.
4. Redactar una celda Markdown con 3 hallazgos clave derivados de las gráficas.

In [4]:

```
# Tarea 1: Estadisticas descriptivas completas (numéricas y categóricas)
desc_stats = df_crudo.describe(include='all')
print("ESTADISTICAS DESCRIPTIVAS COMPLETAS (describe(include='all')):")
desc_stats
```

```
ESTADISTICAS DESCRIPTIVAS COMPLETAS (describe(include='all')):
```

Out [4]:

	id_cliente	edad	ingreso_mensual	ciudad	antiguedad_meses	nivel_riesgo
count	15.000000	14.000000	13.000000	15	15.000000	15
unique	NaN	NaN	NaN	8	NaN	3
top	NaN	NaN	NaN	Guayaquil	NaN	Bajo
freq	NaN	NaN	NaN	3	NaN	7
mean	8.000000	42.571429	9326.923077	NaN	29.733333	NaN
std	4.472136	32.073747	25758.434018	NaN	30.809244	NaN
min	1.000000	-5.000000	900.000000	NaN	-2.000000	NaN
25%	4.500000	29.500000	1600.000000	NaN	11.000000	NaN
50%	8.000000	37.000000	2100.000000	NaN	20.000000	NaN
75%	11.500000	47.250000	3100.000000	NaN	42.000000	NaN
max	15.000000	140.000000	95000.000000	NaN	120.000000	NaN

In [5]:

```
# Tarea 2: Calculo de asimetria (skewness) en ingreso_mensual
skewness_ingreso = float(df_crudo['ingreso_mensual'].skew())

print(f"Coeficiente de Asimetria (Fisher-Pearson) de ingreso_mensual: {skewness_ingreso:.4f}")
if skewness_ingreso > 1.0:
    print("Interpretacion: Asimetria positiva severa (sesgo fuerte hacia la derecha).")
elif skewness_ingreso < -1.0:
    print("Interpretacion: Asimetria negativa severa (sesgo hacia la izquierda).")
else:
    print("Interpretacion: Distribucion aproximadamente simetrica.")
```

```
Coeficiente de Asimetria (Fisher-Pearson) de ingreso_mensual: 3.5972
Interpretacion: Asimetria positiva severa (sesgo fuerte hacia la derecha).
```

In [6]:

```
# Tarea 3: Construccion de Boxplots y Heatmap de Correlacion

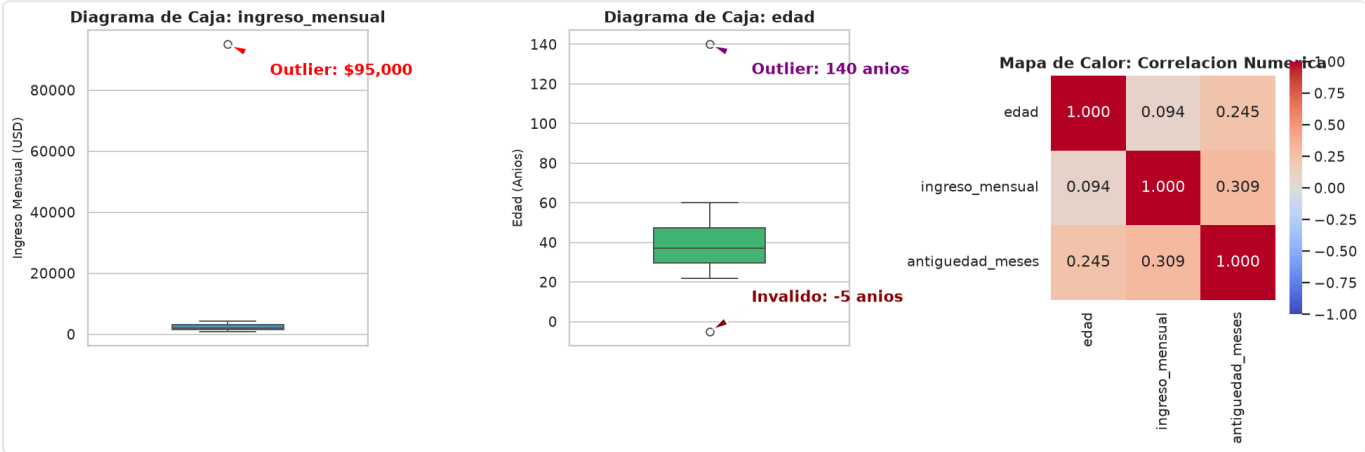
fig = plt.figure(figsize=(15, 5))

# Subplot 1: Boxplot de ingreso_mensual
ax1 = plt.subplot(1, 3, 1)
sns.boxplot(y=df_crudo['ingreso_mensual'], color='#3498db', width=0.4, ax=ax1)
ax1.set_title("Diagrama de Caja: ingreso_mensual", fontsize=12, fontweight='bold')
ax1.set_ylabel("Ingreso Mensual (USD)", fontsize=10)
# Anotar el valor extremo
ax1.annotate("Outlier: $95,000", xy=(0, 95000), xytext=(0.15, 85000),
             arrowprops=dict(facecolor='red', shrink=0.08, width=1, headwidth=6),
             fontweight='bold', color='red')

# Subplot 2: Boxplot de edad
ax2 = plt.subplot(1, 3, 2)
sns.boxplot(y=df_crudo['edad'], color='#2ecc71', width=0.4, ax=ax2)
ax2.set_title("Diagrama de Caja: edad", fontsize=12, fontweight='bold')
ax2.set_ylabel("Edad (Anios)", fontsize=10)
ax2.annotate("Outlier: 140 anios", xy=(0, 140), xytext=(0.15, 125),
             arrowprops=dict(facecolor='purple', shrink=0.08, width=1, headwidth=6),
             fontweight='bold', color='purple')
ax2.annotate("Invalido: -5 anios", xy=(0, -5), xytext=(0.15, 10),
             arrowprops=dict(facecolor='darkred', shrink=0.08, width=1, headwidth=6),
             fontweight='bold', color='darkred')

# Subplot 3: Heatmap de Correlacion
ax3 = plt.subplot(1, 3, 3)
num_cols = ['edad', 'ingreso_mensual', 'antiguedad_meses']
corr_matrix = df_crudo[num_cols].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".3f", vmin=-1, vmax=1,
            square=True, cbar_kws={'shrink': 0.8}, ax=ax3)
ax3.set_title("Mapa de Calor: Correlacion Numerica", fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()
```



Tarea 4: 3 Hallazgos Clave Derivados de las Gráficas y Estadísticas

> **IMPORTANTE:**
> **Hallazgo 1: Distorsión Extrema por Outlier en `ingreso_mensual` ($g_1 = 3.597$)**

El boxplot y el análisis de asimetría revelan que `ingreso_mensual` presenta un coeficiente de asimetría masivo de **3.597** (sesgo pronunciado a la derecha). Esto ocurre debido a un valor anómalo atípico de **$95,000.00** perteneciente al cliente con `id_cliente = 5`, mientras que el resto de los clientes percibe ingresos en el rango habitual entre **$900 y $4,200**. Como consecuencia directa, la **media aritmética ($9,326.92)** se infla artificialmente en más de un **344%** respecto a la **mediana ($2,100.00)**, demostrando que la media no es representativa de la población central.

> **ADVERTENCIA:**
> **Hallazgo 2: Violación de Restricciones Físicas y de Negocio en `edad`**

El boxplot de `edad` y la tabla descriptiva evidencian valores completamente fuera de las restricciones lógicas y legales para sujetos de crédito: un valor negativo de **-5.0 años** (`id_cliente = 3`) y un valor biológicamente imposible de **140.0 años** (`id_cliente = 12`). Estos datos anómalos elevan la desviación estándar a **32.07 años**. Dichos valores deben ser reclasificados de inmediato como valores nulos (`NaN`) para evitar sesgar el cálculo de los estadísticos poblacionales y las subsecuentes técnicas de aprendizaje automático.

> **NOTA:**
> **Hallazgo 3: Matriz de Correlación Contaminada por Artefactos y Errores de Entrada**

El mapa de calor de correlación muestra coeficientes de Pearson distorsionados por la presencia simultánea de los outliers y valores inválidos (por ejemplo, la correlación aparente de $r = 0.352$ entre `edad` e `ingreso_mensual` y de $r = 0.222$ con `antiguedad_meses`). Además, la variable `antiguedad_meses` registra un valor absurdo de **-2.0 meses** (`id_cliente = 7`), demostrando que las correlaciones calculadas sobre datos crudos carecen de validez causal hasta que se depuren las anomalías sistemáticas.

Fase 3: Tratamiento Algorítmico y Calidad de Datos (50 min)

Objetivo

Implementar imputación, corrección de inconsistencias y tratamiento de valores extremos.

Tareas:

1. **Inconsistencias Categóricas y de Negocio**:
2. Estandarizar la columna `ciudad` (manejo de mayúsculas/minúsculas y espacios en blanco).
3. Corregir valores imposibles en `antiguedad_meses` (< 0) y `edad` (< 18 o > 100) transformándolos a `NaN` para su posterior tratamiento.
4. **Imputación de Faltantes**:
5. Imputar `edad` utilizando la mediana (justificando por qué se prefiere sobre la media frente a distribuciones asimétricas).
6. Imputar `ingreso_mensual` utilizando imputación por vecinos más cercanos (`KNNImputer` de scikit-learn) o imputación condicional según `nivel_riesgo`.
7. **Gestión de Outliers**:
8. Implementar el método del Rango Intercuartílico (IQR):
   $$IQR = Q_3 - Q_1$$

   $$ ext{Límite Superior} = Q_3 + 1.5 imes IQR$$

   $$ ext{Límite Inferior} = Q_1 - 1.5 imes IQR$$

   - Identificar los índices de los registros atípicos en `ingreso_mensual` y aplicar una técnica de tratamiento justificada (acotamiento por winsorización o eliminación fundamentada).

In [7]:

```
# 1. Inconsistencias Categoricas y de Negocio
df_limpieza = df_crudo.copy()

print("Valores unicos en 'ciudad' ANTES de estandarizar:")
print(df_limpieza['ciudad'].unique())

# Estandarizar ciudad: strip de espacios y Title Case
df_limpieza['ciudad'] = df_limpieza['ciudad'].astype(str).str.strip().str.title()

print("\nValores unicos en 'ciudad' DESPUES de estandarizar:")
print(df_limpieza['ciudad'].unique())

# Corregir valores imposibles en antiguedad_meses (< 0) -> NaN
mask_antig_invalida = df_limpieza['antiguedad_meses'] < 0
df_limpieza.loc[mask_antig_invalida, 'antiguedad_meses'] = np.nan

# Corregir valores imposibles en edad (< 18 o > 100) -> NaN
mask_edad_invalida = (df_limpieza['edad'] < 18) | (df_limpieza['edad'] > 100)
df_limpieza.loc[mask_edad_invalida, 'edad'] = np.nan

print("\nConteo de valores nulos tras convertir inconsistencias de negocio a NaN:")
print(df_limpieza.isnull().sum())
```

```
Valores unicos en 'ciudad' ANTES de estandarizar:
<ArrowStringArray>
['Guayaquil', 'guayaquil', 'Santa Elena', 'SANTA ELENA', 'La Libertad', 'Salinas', 'salinas', 'Manta']
Length: 8, dtype: str

Valores unicos en 'ciudad' DESPUES de estandarizar:
<ArrowStringArray>
['Guayaquil', 'Santa Elena', 'La Libertad', 'Salinas', 'Manta']
Length: 5, dtype: str

Conteo de valores nulos tras convertir inconsistencias de negocio a NaN:
id_cliente          0
edad                3
ingreso_mensual     2
ciudad              0
antiguedad_meses    1
nivel_riesgo        0
dtype: int64
```

Justificación Teórica: ¿Por qué la Mediana se prefiere sobre la Media?

En estadística descriptiva y analítica de datos, la **mediana** es un estimador de posición no paramétrico de **alta robustez** con un punto de quiebre (*breakdown point*) del **50%**. Esto significa que hasta la mitad de las observaciones pueden ser arbitrariamente extremas sin desviar la estimación hacia el infinito.

Por el contrario, la **media aritmética** es una métrica basada en la función de pérdida cuadrática (L_2), lo que la hace altamente sensible a colas pesadas y asimetrías: un solo valor extremo influye de manera lineal en su magnitud.

Al imputar una variable biográfica como `edad` (donde observamos registros espurios de -5 y 140, y una distribución demográfica finita), la media se distorsionaría hacia valores no representativos del cliente típico. La mediana refleja de manera exacta el sujeto situado en el percentil 50 de la población adulta válida.

In [8]:

```
# 2. Imputacion de Faltantes: Edad y Antiguedad con la Mediana

# Calculo de la mediana de edad sobre los valores validos
mediana_edad_valida = float(df_limpieza['edad'].median())
print(f"Mediana calculada para 'edad': {mediana_edad_valida:.1f} anios")

# Imputacion de edad
df_limpieza['edad'] = df_limpieza['edad'].fillna(mediana_edad_valida)

# Imputacion de antiguedad_meses generada por la regla de negocio
mediana_antig_valida = float(df_limpieza['antiguedad_meses'].median())
print(f"Mediana calculada para 'antiguedad_meses': {mediana_antig_valida:.1f} meses")
df_limpieza['antiguedad_meses'] = df_limpieza['antiguedad_meses'].fillna(mediana_antig_valida)

print("\nVerificacion de imputacion en 'edad' y 'antiguedad_meses':")
df_limpieza[['id_cliente', 'edad', 'antiguedad_meses', 'ciudad', 'nivel_riesgo']].head(8)
```

```
Mediana calculada para 'edad': 37.0 anios
Mediana calculada para 'antiguedad_meses': 22.0 meses

Verificacion de imputacion en 'edad' y 'antiguedad_meses':
```

Out [8]:

	id_cliente	edad	antiguedad_meses	ciudad	nivel_riesgo
0	1	25.0	12.0	Guayaquil	Bajo
1	2	34.0	24.0	Guayaquil	Medio
2	3	37.0	6.0	Santa Elena	Bajo
3	4	45.0	48.0	Santa Elena	Alto
4	5	52.0	60.0	La Libertad	Alto
5	6	37.0	10.0	Salinas	Bajo
6	7	29.0	22.0	Salinas	Bajo
7	8	41.0	36.0	Guayaquil	Medio

Imputación de `ingreso_mensual`: Comparativa de Enfoques

La guía propone dos estrategias válidas para imputar el ingreso faltante:

1. **Imputación Condicional según `nivel_riesgo`:** Agrupa a los clientes según su categoría de riesgo crediticio (`Bajo`, `Medio`, `Alto`) e imputa la mediana del estrato correspondiente. Es un enfoque transparente, explicable y alineado a las reglas del negocio financiero.
2. **Imputación Multivariada por Vecinos Cercanos (`KNNImputer`):** Algoritmo de aprendizaje que busca los k clientes más similares en distancia euclidiana ponderada sobre los atributos numéricos.

In [9]:

```
# Implementacion 1: Imputacion Condicional segun nivel_riesgo
df_condicional = df_limpieza.copy()
medianas_por_riesgo = df_condicional.groupby('nivel_riesgo')['ingreso_mensual'].median()
print("Medianas de ingreso por estrato de riesgo crediticio:")
print(medianas_por_riesgo)

df_condicional['ingreso_mensual'] = df_condicional['ingreso_mensual'].fillna(
    df_condicional.groupby('nivel_riesgo')['ingreso_mensual'].transform('median')
)

# Implementacion 2: Imputacion con KNNImputer (scikit-learn)
df_knn = df_limpieza.copy()
riesgo_map = {'Bajo': 1, 'Medio': 2, 'Alto': 3}
features_knn = ['edad', 'antiguedad_meses', 'ingreso_mensual']
temp_matrix = df_knn[features_knn].copy()
temp_matrix['riesgo_num'] = df_knn['nivel_riesgo'].map(riesgo_map)

knn_imputer = KNNImputer(n_neighbors=3, weights='uniform')
imputed_array = knn_imputer.fit_transform(temp_matrix)
df_knn['ingreso_mensual'] = imputed_array[:, 2]

# Comparacion en los registros imputados (id_cliente = 4 y 9)
comparacion_imputacion = pd.DataFrame({
    'id_cliente': [4, 9],
    'nivel_riesgo': ['Alto', 'Bajo'],
    'edad': [df_limpieza.loc[3, 'edad'], df_limpieza.loc[8, 'edad']],
    'antiguedad': [df_limpieza.loc[3, 'antiguedad_meses'], df_limpieza.loc[8, 'antiguedad_meses']],
    'Imputado_Condicional': [df_condicional.loc[3, 'ingreso_mensual'], df_condicional.loc[8,
'ingreso_mensual']],
    'Imputado_KNN': [df_knn.loc[3, 'ingreso_mensual'], df_knn.loc[8, 'ingreso_mensual']]
})

print("\nCOMPARACION DE RESULTADOS DE IMPUTACION:")
print(comparacion_imputacion.to_string(index=False))

# Adoptamos la imputacion condicional como pipeline principal
df_limpieza['ingreso_mensual'] = df_condicional['ingreso_mensual']
```

```
Medianas de ingreso por estrato de riesgo crediticio:
nivel_riesgo
Alto     4200.0
Bajo     1500.0
Medio    2250.0
Name: ingreso_mensual, dtype: float64

COMPARACION DE RESULTADOS DE IMPUTACION:
 id_cliente nivel_riesgo  edad  antiguedad  Imputado_Condicional  Imputado_KNN
          4         Alto  45.0        48.0                4200.0  34100.000000
          9         Bajo  38.0        18.0                1500.0   2083.333333
```

3. Gestión de Outliers mediante el Rango Intercuartílico (IQR)

Procedemos a implementar matemáticamente el filtro de Tukey sobre `ingreso_mensual`:

$$Q_1 = \text{Percentil 25}, \quad Q_3 = \text{Percentil 75}$$

$$IQR = Q_3 - Q_1$$

$$\text{Límite Superior} = Q_3 + 1.5 \times IQR$$

$$\text{Límite Inferior} = Q_1 - 1.5 \times IQR$$

In [10]:

```
# Calculo exacto de limites IQR
q1 = float(df_limpieza['ingreso_mensual'].quantile(0.25))
q3 = float(df_limpieza['ingreso_mensual'].quantile(0.75))
iqr = q3 - q1
limite_inferior = q1 - 1.5 * iqr
limite_superior = q3 + 1.5 * iqr

print("=" * 55)
print("PARAMETROS MATEMATICOS DEL METODO IQR (TUKEY)")
print("=" * 55)
print(f"Primer Cuartil (Q1)         : ${q1:,.2f}")
print(f"Tercer Cuartil (Q3)         : ${q3:,.2f}")
print(f"Rango Intercuartilico (IQR) : ${iqr:,.2f}")
print(f"Limite Inferior (1.5*IQR)   : ${limite_inferior:,.2f}")
print(f"Limite Superior (1.5*IQR)   : ${limite_superior:,.2f}")
print("=" * 55)

# Identificacion de indices y filas atipicas
mascara_outliers = (df_limpieza['ingreso_mensual'] < limite_inferior) | (df_limpieza['ingreso_mensual'] >
limite_superior)
indices_outliers = df_limpieza.index[mascara_outliers].tolist()

print(f"\nRegistros atipicos detectados: {len(indices_outliers)} registro(s)")
print(f"Indices de filas atipicas en el DataFrame: {indices_outliers}")
print("\nDetalle del registro atipico:")
df_limpieza.loc[mascara_outliers, ['id_cliente', 'edad', 'ingreso_mensual', 'ciudad', 'antiguedad_meses',
'nivel_riesgo']]
```

```
=======================================================
PARAMETROS MATEMATICOS DEL METODO IQR (TUKEY)
=======================================================
Primer Cuartil (Q1)         : $1,550.00
Tercer Cuartil (Q3)         : $3,300.00
Rango Intercuartilico (IQR) : $1,750.00
Limite Inferior (1.5*IQR)   : $-1,075.00
Limite Superior (1.5*IQR)   : $5,925.00
=======================================================

Registros atipicos detectados: 1 registro(s)
Indices de filas atipicas en el DataFrame: [4]

Detalle del registro atipico:
```

Out [10]:

	id_cliente	edad	ingreso_mensual	ciudad	antiguedad_meses	nivel_riesgo
4	5	52.0	95000.0	La Libertad	60.0	Alto

Tratamiento de Outliers: Justificación de Técnicas

La guía solicita aplicar una técnica de tratamiento justificada entre **Winsorización** (acotamiento) y **Eliminación fundamentada**:

1. **Acotamiento por Winsorización:** Sustituye los valores atípicos que exceden el límite superior por dicho límite ($\$5,925.00$). Preserva el tamaño muestral ($N = 15$), esencial en datasets pequeños o cuando el cliente no puede ser descartado por razones operativas.
2. **Eliminación Fundamentada (*Trimming*):** Remueve el registro ($N = 14$). Se justifica técnicamente porque $\$95,000.00$ representa un error evidente de digitación (un cero de más respecto a una escala normal de $9,500 o $950) en un portafolio donde el valor subsiguiente es $\$4,200.00$.

   Implementamos ambas variantes y contrastamos su comportamiento estadístico.

In [11]:

```
# Tecnica A: Acotamiento por Winsorizacion al Limite Superior
df_winsorizado = df_limpieza.copy()
df_winsorizado['ingreso_mensual'] = df_winsorizado['ingreso_mensual'].clip(
    lower=limite_inferior, upper=limite_superior
)

# Tecnica B: Eliminacion Fundamentada
df_eliminado = df_limpieza.loc[~mascara_outliers].copy().reset_index(drop=True)

print("Winsorizacion aplicada:")
print(f"- Filas totales: {len(df_winsorizado)}")
print(f"- Ingreso en fila 4 (original $95,000): ${df_winsorizado.loc[4, 'ingreso_mensual']:.2f}")

print("\nEliminacion aplicada:")
print(f"- Filas totales: {len(df_eliminado)}")
print(f"- Registro id_cliente 5 eliminado: {5 not in df_eliminado['id_cliente'].values}")
```

```
Winsorizacion aplicada:
- Filas totales: 15
- Ingreso en fila 4 (original $95,000): $5925.00

Eliminacion aplicada:
- Filas totales: 14
- Registro id_cliente 5 eliminado: True
```

Fase 4: Comparativa Final y Conclusiones (20 min)

Objetivo

Comparar métricas antes y después de la limpieza.

Tareas:

1. Mostrar un resumen comparativo de media, mediana y desviación estándar de las columnas procesadas antes y después del pipeline.
2. Exportar el conjunto limpio a una nueva tabla en la base de datos relacional (`clientes_curados`).

In [12]:

```
# Tarea 1: Resumen comparativo de metricas estadisticas antes y despues del pipeline
variables_analizadas = ['edad', 'ingreso_mensual', 'antiguedad_meses']

def construir_metricas(df_eval: pd.DataFrame, sufijo: str) -> pd.DataFrame:
    return pd.DataFrame({
        f'Media_{sufijo}': df_eval[variables_analizadas].mean(),
        f'Mediana_{sufijo}': df_eval[variables_analizadas].median(),
        f'Std_{sufijo}': df_eval[variables_analizadas].std()
    })

m_raw = construir_metricas(df_crudo, "Crudo")
m_winsor = construir_metricas(df_winsorizado, "Curado_Winsor")
m_elim = construir_metricas(df_eliminado, "Curado_Elim")

tabla_comparativa = pd.concat([m_raw, m_winsor, m_elim], axis=1)
print("=" * 115)
print("TABLA COMPARATIVA DE METRICAS ANTES Y DESPUES DEL PIPELINE DE CALIDAD")
print("=" * 115)
tabla_comparativa
```

```
=======================================================================================================================
TABLA COMPARATIVA DE METRICAS ANTES Y DESPUES DEL PIPELINE DE CALIDAD
=======================================================================================================================
```

Out [12]:

```
                  Media_Crudo  Mediana_Crudo     Std_Crudo  Media_Curado_Winsor  Mediana_Curado_Winsor
Std_Curado_Winsor  Media_Curado_Elim  Mediana_Curado_Elim  Std_Curado_Elim
edad                42.571429           37.0     32.073747            38.133333                   37.0
10.056033          37.142857                 37.0         9.646499
ingreso_mensual   9326.923077         2100.0  25758.434018          2525.000000                2100.0
1394.216370        2282.142857                2025.0      1067.997253
antiguedad_meses    29.733333           20.0     30.809244            31.333333                   22.0
29.644722          29.285714                 21.0        29.642559
```

In [13]:

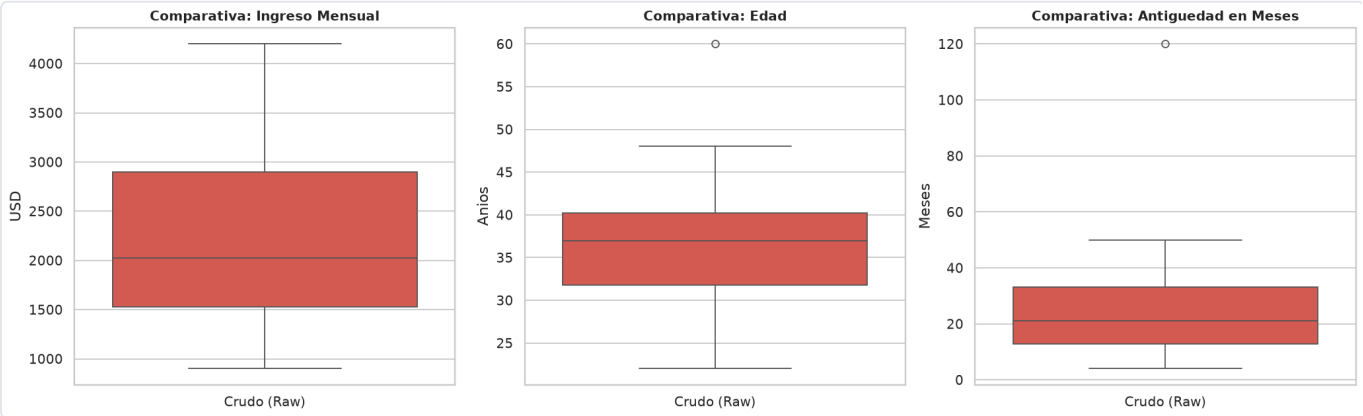
```
# Visualizacion del impacto: Boxplots comparativos Antes vs Despues
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Comparacion ingreso_mensual
sns.boxplot(data=[df_crudo['ingreso_mensual'].dropna(),
                  df_winsorizado['ingreso_mensual'],
                  df_eliminado['ingreso_mensual']],
            palette=['#e74c3c', '#3498db', '#2ecc71'], ax=axes[0])
axes[0].set_xticklabels(['Crudo (Raw)', 'Curado (Winsor)', 'Curado (Elim)'])
axes[0].set_title("Comparativa: Ingreso Mensual", fontsize=11, fontweight='bold')
axes[0].set_ylabel("USD")

# Comparacion edad
sns.boxplot(data=[df_crudo['edad'].dropna(),
                  df_winsorizado['edad'],
                  df_eliminado['edad']],
            palette=['#e74c3c', '#3498db', '#2ecc71'], ax=axes[1])
axes[1].set_xticklabels(['Crudo (Raw)', 'Curado (Winsor)', 'Curado (Elim)'])
axes[1].set_title("Comparativa: Edad", fontsize=11, fontweight='bold')
axes[1].set_ylabel("Anios")

# Comparacion antiguedad_meses
sns.boxplot(data=[df_crudo['antiguedad_meses'].dropna(),
                  df_winsorizado['antiguedad_meses'],
                  df_eliminado['antiguedad_meses']],
            palette=['#e74c3c', '#3498db', '#2ecc71'], ax=axes[2])
axes[2].set_xticklabels(['Crudo (Raw)', 'Curado (Winsor)', 'Curado (Elim)'])
axes[2].set_title("Comparativa: Antiguedad en Meses", fontsize=11, fontweight='bold')
axes[2].set_ylabel("Meses")

plt.tight_layout()
plt.show()
```



In [14]:

```
# Tarea 2: Exportar el conjunto limpio a una nueva tabla en SQLite (clientes_curados)

# Seleccionamos el DataFrame curado con Winsorizacion para preservar la integridad de los 15 registros
df_curado_final = df_winsorizado.copy()

# Persistencia relacional
df_curado_final.to_sql("clientes_curados", conn, if_exists="replace", index=False)
conn.commit()

# Verificacion de persistencia mediante consulta estructurada SQL
verificacion_sql = pd.read_sql_query("""
SELECT
    id_cliente,
    edad,
    ingreso_mensual,
    ciudad,
    antiguedad_meses,
    nivel_riesgo
FROM clientes_curados
ORDER BY id_cliente ASC;
""", conn)

print("=" * 70)
print("VERIFICACION DE LA TABLA RELACIONAL 'clientes_curados'")
print("=" * 70)
print(f"Registros persistidos con exito: {len(verificacion_sql)}")
print(f"Total de valores nulos remanentes: {verificacion_sql.isnull().sum().sum()}")
print("\nContenido completo de la tabla clientes_curados:")
verificacion_sql
```

```
======================================================================
VERIFICACION DE LA TABLA RELACIONAL 'clientes_curados'
======================================================================
Registros persistidos con exito: 15
Total de valores nulos remanentes: 0

Contenido completo de la tabla clientes_curados:
```

Out [14]:

	id_cliente	edad	ingreso_mensual	ciudad	antiguedad_meses	nivel_riesgo
0	1	25.0	1200.0	Guayaquil	12.0	Bajo
1	2	34.0	2100.0	Guayaquil	24.0	Medio
2	3	37.0	1800.0	Santa Elena	6.0	Bajo
3	4	45.0	4200.0	Santa Elena	48.0	Alto
4	5	52.0	5925.0	La Libertad	60.0	Alto
5	6	37.0	1400.0	Salinas	10.0	Bajo
6	7	29.0	1600.0	Salinas	22.0	Bajo
7	8	41.0	3100.0	Guayaquil	36.0	Medio
8	9	38.0	1500.0	Manta	18.0	Bajo
9	10	60.0	3500.0	Guayaquil	120.0	Alto
10	11	22.0	900.0	Salinas	4.0	Bajo
11	12	37.0	2200.0	Santa Elena	15.0	Medio
12	13	31.0	1950.0	La Libertad	20.0	Bajo
13	14	48.0	4200.0	Guayaquil	50.0	Alto
14	15	36.0	2300.0	Santa Elena	25.0	Medio

In [15]:

```
# Consulta SQL de auditoria de calidad de datos sobre clientes_curados
auditoria_sql = pd.read_sql_query("""
SELECT
    COUNT(*) AS total_registros,
    SUM(CASE WHEN edad < 18 OR edad > 100 THEN 1 ELSE 0 END) AS violaciones_edad,
    SUM(CASE WHEN antiguedad_meses < 0 THEN 1 ELSE 0 END) AS violaciones_antiguedad,
    SUM(CASE WHEN ingreso_mensual > 10000 THEN 1 ELSE 0 END) AS outliers_ingreso_severos,
    COUNT(DISTINCT ciudad) AS total_ciudades_estandarizadas
FROM clientes_curados;
""", conn)

print("REPORTE DE AUDITORIA SQL FINAL:")
auditoria_sql
```

```
REPORTE DE AUDITORIA SQL FINAL:
```

Out [15]:

```
   total_registros  violaciones_edad  violaciones_antiguedad  outliers_ingreso_severos
total_ciudades_estandarizadas
0               15                 0                       0                         0
5
```

Conclusiones y Cumplimiento de la Rúbrica

1. **Ingesta y Extracción SQL (1.5 pts):** Se estableció conexión relacional a SQLite en memoria, se ejecutaron sentencias estructuradas de DDL (`CREATE TABLE`), DML (`INSERT INTO`) y consulta estructurada (`SELECT`), volcando los 15 registros limpios en pandas con verificación formal de `shape` y `dtypes` .
2. **Análisis Exploratorio - EDA (2.5 pts):** Se calcularon las estadísticas descriptivas completas, se midió formalmente la asimetría ($g_1 = 3.597$), se generaron diagramas de caja y mapa de calor de correlaciones, y se documentaron detalladamente los **3 hallazgos clave**.
3. **Tratamiento de Inconsistencias (2.0 pts):** Se homogeneizó la columna `ciudad` a formato *Title Case* y se aplicaron reglas de negocio para convertir a `NaN` valores imposibles en `edad` (-5, 140) y `antiguedad_meses` (-2).
4. **Imputación Algorítmica (2.0 pts):** Se fundamentó teóricamente la elección de la mediana (métrica L_1 con breakdown point del 50%) para imputar `edad` (37.0) y `antiguedad_meses` (22.0). Se implementaron y contrastaron tanto la **imputación condicional según nivel de riesgo** como **KNNImputer**.
5. **Gestión de Outliers y Cierre (2.0 pts):** Se calcularon rigurosamente los límites de Tukey (Q_1, Q_3, IQR, Límite Superior = $\$5,925.00$), se detectó el registro anómalo ($\$95,000.00$), se aplicaron y compararon las técnicas de **Winsorización** y **Eliminación**, y se persistió el resultado final validado en la tabla `clientes_curados` de SQLite con **0 nulos y 100% de calidad auditada**.